

第5章 函数

5.1 函数声明和定义

广义的函数声明，包括函数原型和函数定义；狭义的函数声明一般特指函数原型或函数签名 (Signature)，对于函数定义，通常也称函数实现。它们的常见语法格式为：

函数原型格式：

```
[extern "C" | extern "C++"] 返回类型 [调用约定] 函数名( [参数表]) [const];
```

函数定义格式：

```
[ extern ["C" | "C++"] ] 返回类型 [调用约定] 函数名( [参数表]) [const]
{
    具体实现
}
```

`extern` 关键字表示该函数具有外部链接性，意味着它可以被其他编译单元或程序调用，这些编译单元或程序甚至可以用其他语言编写。在编译阶段中，带有 `extern` 修饰的函数会被列入目标文件的外部符号表，从而使得其他编译单元在链接阶段能够访问该函数。

`extern "C"` 表示函数采用 C 语言风格的外部链接方式；而 `extern "C++"` 则表示函数采用 C++ 语言风格的外部链接方式，这也是 C++ 中非成员函数的默认链接方式，因此通常无需显式指定。

这里我们给出的函数语法格式只是最常见的形式，实际上函数还可以有其它的形式和修饰，如异常说明，`final` 修饰，缺省实现等，这些会在后续内容中给出说明。有关调用约定请参考函数的调用约定，有关函数的 `const` 修饰请参考类的成员，有关异常说明请参考异常的处理，有关 `final` 修饰请参考虚机制。

以下均为合法的函数声明：

```
1 void buy(int num);           //C++98 下，默认__cdecl 调用，C++11 以上，默认__stdcall 调用
2 int __cdecl myFunc( int );    //标准 C 调用；形参名可省略
3 int __stdcall sell(Sheep & aSheep) ; //__stdcall 调用
4 int & max2(int & one,int & ) throw( ); //C++98,不会抛出任何异常
5 int & max2(int & one,int & ) noexcept; //C++11 以上,不会抛出任何异常
6 extern "C" {
7     int f( int );
8     void g( int );
9 }
```

函数的声明和定义也需遵循与变量类似的原则：先声明后使用，并且遵守单一定义规则。也就

是说，在一个编译单元中，调用的任何函数都必须事先声明，而且在整个项目的多个编译单元中一个函数最多只能有一个实现。

有了函数定义，可以唯一地确定函数的入口地址。但有两点还需要注意：

1. 函数的返回类型，不能作为函数的区分标识。例如，下边三个函数至多能存在一个。

```
1 void f( ) { ... }
2 int f( ) { ... }
3 A f( ) { ... }
```

2. 函数的形参名，不作为函数的区分标识，甚至形参名有无均可。例如，下边的代码是合法的。

```
1 int f(int a,int b);
2 int f(int,int);
3 int f(int c,int) {
4     return c;
5 }
6 int main( ) {
7     int var = 100;
8     return f(var,4);
9 }
```

5.2 函数的调用过程

为了更好地理解函数的调用过程，下面以给定的程序代码为例^[38]，分析其在 Windows 平台、X86 处理器及 32 位模式下的汇编结果，并进行简要说明。程序源代码如下：

```
1 #include <stdio>
2 void __cdecl func(int param1, int param2, int param3)
3 {
4     int var1 = param1;
5     int var2 = param2;
6     int var3 = param3;
7     ...
8     return ;
9 }
10 int main( ) {
11     func(1, 2, 3);
12     return 0;
13 }
```

以上源代码的汇编码和简要说明如下：

```

1  11: int main( ) {
2  .....; 省略了建立堆栈的代码
3  12:  func(1, 2, 3);
4  00401118 push    3                ; 将 param3 压入栈
5  0040111A push    2                ; 将 param2 压入栈
6  0040111C push    1                ; 将 param1 压入栈
7  0040111E call    @ILT+5(func) (0040100a); @ILT+5(func) 是函数 func 的修饰名, 而 0040100a
则是它的地址
8  00401123 add     esp,0Ch ; 恢复堆栈, __cdecl 规定由调用者(这里是 main)恢复堆栈
9  13:  return 0;
10 00401126 xor     eax,eax
11 14:  }
12 00401128 pop     edi
13 00401129 pop     esi
14 0040112A pop     ebx
15 0040112B add     esp,40h
16 0040112E cmp     ebp,esp
17 00401130 call    __chkesp (004011d0)
18 00401135 mov     esp,ebp
19 00401137 pop     ebp
20 00401138 ret

```

从上边的汇编码, 可以看到:

- (1) 4-6 行, 表示参数压栈, 而且是从右到左。
- (2) 8 行, 恢复栈的操作, 实际只是修改栈顶指针的位置, 即增加栈顶指针寄存器中的数值。
- (3) 10 行, 记录返回值, 返回值存放在 `eax` 寄存器。
- (4) 12-20 行, 恢复调用现场, 就是将相关寄存器中值恢复到调用前的值。

```

1  void __cdecl func(int param1, int param2, int param3) {
2  00401020 push    ebp
3  00401021 mov     ebp,esp
4  00401023 sub     esp,4Ch
5  00401026 push    ebx
6  00401027 push    esi
7  00401028 push    edi
8  00401029 lea     edi,[ebp-4Ch]
9  0040102C mov     ecx,13h
10 00401031 mov     eax,0CCCCCCCCh
11 00401036 rep stos dword ptr [edi]
12 ; 注意 var1, var2, var3 压入堆栈的顺序!
13 4:  int var1 = param1;
14 00401038 mov     eax,dword ptr [ebp+8]
15 0040103B mov     dword ptr [ebp-4],eax

```

```

16  5:    int var2 = param2;
17  0040103E mov     ecx,dword ptr [ebp+0Ch]
18  00401041 mov     dword ptr [ebp-8],ecx
19  6:    int var3 = param3;
20  00401044 mov     edx,dword ptr [ebp+10h]
21  00401047 mov     dword ptr [ebp-0Ch],edx
22  .....; 省略了...的代码
23  8:    return ;
24  9:}
25  004010BD pop     edi
26  004010BE pop     esi
27  004010BF pop     ebx
28  004010C0 add     esp,4Ch
29  004010C3 cmp     ebp,esp
30  004010C5 call    __chkesp (004011d0)
31  004010CA mov     esp,ebp
32  004010CC pop     ebp
33  004010CD ret     ; 这里是 ret, 由调用者(main)恢复堆栈, 但如果是 __stdcall 的话, 恢复堆栈就
    在这里进行

```

从上边的汇编码, 可以看到:

- (1) 2-11 行, 记录调用现场, 即将当前寄存器的值保存到栈中。
- (2) 13-21 行, 在栈中创建 3 个局部变量。
- (3) 25-33 行, 同 main 函数的汇编码中的 10-20 行, 这里只是为了说明。如果是 `__stdcall` 的话, 会在被调用函数中恢复堆栈。

虽然上面的代码只是一个简单的示例, 但我们从中可以得出几个重要结论:

- 栈的存在是实现过程式编程的基础。
- 栈中只存放调用现场信息和程序数据(注意 `push` 语句的内容), 不会存放可执行代码。
- 参数按从右到左的顺序压栈是因为历史原因: 栈的设计是从高端地址向低端地址生长。这样从右到左压栈可以确保第一个参数的地址最低, 最后一个参数的地址最高, 这符合大多数程序员的理解习惯, 便于调试程序。
- 恢复栈的责任最好由被调用者承担, 但这要求被调用者必须知道参数的类型和数量。对于变长参数的情况, 只能由调用者负责恢复栈, 例如 `printf` 函数。
- 函数调用是一种耗费系统资源的操作, 尤其是当一个函数被频繁调用时, 即便其实现非常简单, 也需要执行记录和恢复调用现场的操作。

5.3 函数的调用约定

上一节中的代码是针对 X86-32 位架构的一个示例。实际上，源程序在不同操作系统中编译成二进制程序后，会在不同的体系结构(如 X86、PowerPC、ARM、RISC-V 等)上执行。为此，生成的二进制程序必须遵循特定体系结构的应用程序二进制接口(Application Binary Interface - ABI)规范。这些规范涵盖了目标文件格式、可执行文件格式、调用约定以及动态库格式等方面的要求。

各操作系统的程序也必须严格遵守相应的 ABI 规范。例如，在 Windows 操作系统中，PE(Portable Executable)格式是常见的可执行文件格式，包括 EXE、DLL、SYS 和 COM 等文件类型。ABI 中关于调用约定(Calling Convention)的要求主要涉及以下几个方面：

- (1) 参数传递方式：参数是通过栈存储还是寄存器存储？
- (2) 参数处理顺序：参数是按从右到左还是从左到右的顺序处理？
- (3) 栈的清理与现场恢复：清栈及恢复调用现场是由调用者负责还是由被调用者负责？
- (4) 返回值存放位置：函数的返回值存放在哪个寄存器中？
- (5) 不同平台的要求：32 / 64 位平台上的要求有哪些不同？

下表简要列出了在 Windows 平台上不同调用约定的差异。其他平台的情况大致相似。

表 5-1 Windows 平台中的调用约定

调用约定	简要说明	栈的清理者	压栈顺序	函数名表示 (不同编译器有细微差别)
<code>__cdecl</code>	C 调用	调用者	从右至左	<code>_MyFuncName</code>
<code>__stdcall</code>	标准调用	被调用者	从右至左	<code>MyFuncName</code>
<code>__pascal</code>	PASCAL 调用	被调用者	从左至右	<code>MYFUNCNAME</code>
<code>__fastcall</code>	前两个参数尽可能放入寄存器	被调用者	从左到右 (VC6.0) 从右到左 (BCB6.0)	<code>@MyFuncName</code>
<code>__thiscall</code>	成员函数的默认调用约定。 <code>this</code> 指针放入 ECX 寄存器，其它默认同 <code>__cdecl</code>	调用者	从右至左	<code>_MyFuncNameXXX</code>

我们常用的调用约定是 `__cdecl` 和 `__stdcall`。当参数按照从右到左的顺序压栈时，对于常见的 X86 架构下的内存模式，这种方式便于调试。`__cdecl` 指明由调用者负责清栈，适用于处理固定参数和变长参数的函数；而 `__stdcall` 指明由被调用者负责清栈，仅适用于处理固定参数的函数。函数在执行过程中是通过地址访问的，而不是依赖函数名。不同的函数名主要用于编译时不同编译单元之间的链接，以及外部应用程序通过函数名来调用函数。

5.4 函数的重载

在 C 语言中,函数名必须唯一。然而,在 C++和现代程序设计语言中,允许存在多个同名函数,只要它们的参数类型、参数顺序、参数个数、const 修饰不完全相同,这被称为**函数重载(overload)**。例如,下面的多个 func 函数就属于函数重载。

```
1  int func( );
2  int func(int);
3  int func(int,int);
4  int func(int,int) const; //只能是成员函数
5  int func(MyClass obj);
```

5.4.1 函数重载条件

函数重载需满足以下条件:

- (1) 函数参数的个数、类型、顺序、是否使用 const 修饰等不完全相同;
- (2) 返回值类型不能作为区分标准;
- (3) 缺省参数也不能作为区分标准;
- (4) 值类型参数的 const 和非 const 形式不作为区分标准;
- (5) 对于引用和指针类型的参数,能否修改实参对象可以作为区分标准;

下面代码中的同名函数属于合法的函数重载:

```
1  void fun1( );
2  void fun1( ) const;
3  int fun2(int);
4  int fun2(int *);
5  int fun3(int & );
6  int fun3(const int &);
7  int fun4(int *);
8  int fun4(const int *);
9  int fun4(int *,int = 6);
```

下面代码中的同名函数属于不合法的函数重载:

```
1  //不合法的重载
2  int fun1( );
3  void fun1( );
4  int fun2(int);
5  int fun2(int& );
6  int fun3(int);
7  int fun3(int = 5);
```

```

8  int fun4(int);
9  int fun4(const int);
10 int fun5(int *);
11 int fun5(int* const);

```

下面的程序演示了重载函数的定义和使用：

```

1  #include <iostream>
2
3  struct A { int x; int y; };
4
5  //void f( const A aA )    { }//非法重载。5 行和 6 行只能留一个
6  void f( A  aA )          { std::cout << "f( A )" << std::endl; }
7  void g( A & aA )         { std::cout << "g( A & )" << std::endl; }
8  void g( const A & aA )   { std::cout << "g( const A & )" << std::endl; }
9  void h( A * pA )         { std::cout << "h( A * )" << std::endl; }
10 void h( const A * pcA )  { std::cout << "h( const A * )" << std::endl; }
11 int main( ) {
12     A a { 1,2 };
13     const A & cA = a;
14     const A * pcA = &a;
15     A * const cpA = &a;
16     const A * const cpcA = &a;
17
18     f( a );           //输出 f( A )
19     f( cA );          //输出 f( A )
20     g( a );           //输出 g( A & )
21     g( cA );          //输出 g( const A & )
22     h( &a );          //输出 h( A * )
23     h( pcA );         //输出 h( const A * )
24     h( cpA );         //输出 h( A * )
25     h( cpcA );        //输出 h( const A * )
26 }

```

对于函数 f，参数是值类型，此时函数是否修改形参的值不会影响实参，对调用者来说，这时调用函数总是安全的，所以形参是否带 `const`，此时不能做区分标识；对于函数 g 和 h，参数是指针或引用类型，是否能修改实参对象关系到函数调用的安全性，此时函数调用会依照安全性匹配适当的重载函数，所以此时可以重载。如果此时有多个重载函数都可以匹配函数的调用，那么编译器会报二义性错误，但这不是函数重载的问题，而是调用方的使用问题。

5.4.2 名字重整

每个函数都有唯一的入口地址。当调用重载函数时，系统内部必须有一套机制来确定具体调用

了哪个版本的重载函数。实际上，在编译阶段会进行一个称为“名字重整”(Name Mangling)的过程。此过程根据特定算法将函数名、参数列表、const 修饰符、异常说明等信息组合成一个新的唯一内部函数名。例如，对于函数 `int func(int)`，在编译后其内部表示可能是 `func_n`；而 `int func( )` 则可能被表示为 `func_v`。通过名字重整可以确保编译后的程序中不会有真正的同名函数。还需要注意的是，不同的编译器采用的名字重整算法各不相同，因此生成的内部函数名也会有所差异。

在同一个项目内，由于使用相同的名字重整算法，编译器可以准确地区分不同的重载函数，不会造成混淆。但是，当外部程序尝试调用另一个程序中的重载函数时，经过名称重整后的函数名变得难以辨认，并且不同编译器生成的名字也不一致，这可能会引发兼容性问题。为了解决这一问题，在 C++ 中可以通过 `extern "C"` 指令显式地禁用名字重整，从而保证函数名的一致性和可预测性。例如：

```
1  extern "C" {
2      int func( );
3      int func(int); //非法
4      int __stdcall otherFun( );
5  }
6  extern "C" void myFunc(int);
7  extern "C" int yourFunc(int,int);
```

这里的 `extern "C"` 明确告知编译器按照 C 语言的风格处理函数，即不进行名字重整，函数的内部名称将直接使用如 `func` 或 `__func` 这样的 C 风格名称。这同时也意味着禁止了函数重载。这种用法常见于多语言混合编程和创建动态链接库。

5.5 函数的参数表

5.5.1 参数的缺省值

在函数的参数列表中，可以明确指定参数的类型、顺序、默认值和形参名称等信息。如果一个函数没有参数，建议使用 `void` 显式声明空的参数列表。例如：

```
1  int f( );    //也可
2  int f(void); //推荐
```

参数的缺省值(也称默认值)是可选的。然而，一旦指定了某个参数的缺省值，那么在此之后的所有参数都必须具有缺省值。例如：

```
1  int f( int param1, int param2 = 3, char = 'h', float = 2.0f);
```


函数参数匹配时，会严格按照实参的顺序依次与形参进行匹配，后续缺少的实参会自动使用缺省值。实际上，在一些现代编程语言中，由于引入了命名参数，对默认值的要求比 C++ 更为宽松。例如，某些语言允许使用如下语法：

```
1 //函数定义
2 int f( int param1 ,int param2 = 2,int param3,int param4 = 4) { /* 略 */}
3 //函数调用
4 int result = f(param1:3, param3:88, param4:99); // 实际会调用 f(3,2,88,99)
```

对于函数的缺省参数，我们需要明确，缺省参数不能用于区分重载函数，并且缺省值的匹配是在编译期间完成的，而不是在运行时进行。例如：

```
1 //a.cpp
2 void func(int n = 6) { cout<<n; }
3 void k( );
4 int main( ) {
5     func( ); //编译为 func(6)
6     k( );
7     return 0;
8 }
9 //b.cpp
10 void func(int = 8);
11 void k( ) {
12     func( ); //编译为 func(8)
13 }
```

显然，上述代码的风格存在坏味道。为了保持函数参数缺省值的一致性，最直接的方法是在一个地方统一指定参数的缺省值。具体做法如下：如果存在函数原型，则应在函数原型中指定参数的缺省值，在函数定义时不再重复指定；该函数原型应放置在头文件中，确保缺省值仅在头文件中出现。只有在没有函数原型的情况下，才允许在函数定义中指定参数的缺省值。针对上述代码，可以做如下修改：

```
1 //func.h
2 #pragma once
3 void func(int n = 7);
4 //a.cpp
5 #include "func.h"
6 void func(int n ) { cout<<n; } //定义时，不再出现缺省值
7 void k( );
8 int main( ) {
9     func( ); //编译成 func(7)
10    k( );
```

```

11     return 0;
12 }
13 //b.cpp
14 #include "func.h"
15 void k( ) {
16     func( ); //编译成 func(7)
17 }

```

5.5.2 形参与实参的匹配

在虚实结合的过程中, 参数的存储位置(栈存储或寄存器存储)和压栈顺序(从左到右或从右到左)由调用约定决定。然而, C++标准并未规定各参数的计算顺序, 因此对于以下代码, 不同编译器的执行结果可能会有所不同。

```

1  int a = 10;
2  f(++a, ++a, ++a);           //压栈的三个参数值, 随编译器不同而不同
3  func( myF(a), yourF(a) + 6 ); //当 myF 和 yourF 有副作用时, 结果多样

```

如果实参与形参的类型不完全匹配, 可能需要进行类型提升或隐式类型转换, 但这种转换有以下限制: 每个参数最多只能进行一次隐式的类型转换。如果一次类型转换后仍然不匹配, 则会直接导致编译错误。例如:

```

1  class MyClass {
2  public:
3      MyClass(int ) { }
4  };
5  class YourClass {
6  public:
7      YourClass( const MyClass &) { }
8  };
9  void f( MyClass obj) { ... }
10 void g( YourClass obj) { ... }
11 int main( ) {
12     f(1); //合法
13     g(2); //非法
14 }

```

在函数 `f(1)` 中, 实参是整数 `1`, 而形参类型为 `MyClass`, 二者不完全匹配。编译器会尝试将整数转换为 `MyClass` 类型, 在本例中存在唯一的转换方式。而在 `g(2)` 中, 实参是整数 `2`, 形参类型为 `YourClass`, 同样不完全匹配。虽然存在从整型到 `MyClass` 再到 `YourClass` 的转换链, 但编译器不允许两次及以上的隐式类型转换。这种限制确保了从一个类型转换到另一个类型时结果的唯一

性和明确性。

参数传递的方式包括**传值**、**传指针**、**传数组**和**传引用**等几种。

在**传值**时，形参是实参在栈区创建的副本，因此在函数内部对形参的修改不会影响原始的实参。

下面的代码是合法的：

```
1  int a = 10;
2  const int b=20;
3  My mObj;
4  const My cmObj;
5  void f(int ,My); 或 void f(const int,const My);
6  f(a,mObj);    //合法
7  f(b,mObj);    //合法
8  f(a,cmObj);   //合法
9  f(b,cmObj);   //合法
```

传递**指针**时，形参在栈区创建的也是实参的副本，但这个副本是一个指针。因此，对指针所指向内容的修改会直接影响实参指针指向的内容。所以，在传递指针时，形参必须满足实参的 **const** 约束。例如：

```
1  int a =10;
2  My mObj;
3  int * pa = &a;
4  My * pmy = &mObj;
5  const int * cpa = pa;
6  const My * cpmy = pmy;
7  void f(int * ,My *);           //重载函数(1)
8  void f(const int *,const My *); //重载函数(2)
9  f(pa,pmy);    //对应(1)
10 f(pa,cpmy);   //对应(2)
11 f(cpa, pmy);  //对应(2)
12 f(cpa,cpmy); //对应(2)
```

传递**数组**时，由于数组只是一种语法概念，在底层实际上是通过指针实现的，因此传递数组本质上是传递指针。所以，对于数组 `int a[3]={1,2,3};`，以下几种方式是等价的。因此，在传递数组时，通常建议增加一个参数来表示数组的长度。

```
1  void f(int[ ]);
2  void f(int a[ ]);
3  void f(int a[3]);
4  void f(int a[5]);
5  void f(int * a);
```

传引用时，由于引用也是语法层面的概念，而其底层实际上是通过指针实现的。因此，传引用本质上还是传指针。所以，在传引用时，形参也必须满足实参的 `const` 约束。例如，若 `int a=1; const int &b = a; const int c=2;`，那么有表 5-2 的结果：

表 5-2 `const` 约束对引用型参数的影响示例

函数原型	合法调用	非法调用
<code>void f(int, int);</code>	<code>f(a,a);f(a,b);f(a,c);f(a,3);</code>	
<code>void f(int&, int&);</code>	<code>f(a,a);</code>	<code>f(a,b); f(a,c);f(a,3);</code>
<code>void f(int, const int&);</code>	<code>f(a,a);f(a,b); f(a,c);f(a,3);f(b,3);</code>	
<code>void f(int&, const int&);</code>	<code>f(a,a); f(a,b); f(a,c); f(a,3);</code>	<code>f(b,b);</code>
<code>void f(const int&, const int&);</code>	<code>f(a,a); f(a,b); f(a,c); f(a,3); f(b,b); f(3,3);</code>	

5.6 函数的返回值

函数的返回值可以有多种形式：

第一种，返回 `void`。例如 `void func()` 表示该函数不返回任何值，此时在函数实现中可以省略 `return` 语句。对于非 `void` 类型的返回值，除非是 `main` 函数，否则必须包含 `return` 语句。

第二种，按值返回。例如 `T func()`，这里的 `T` 可以是内置类型(如基本类型及其衍生类型)或自定义类型。如果 `T` 是内置类型，则返回值隐含为 `const`，例如 `unsigned int f()` 等价于 `const unsigned int f()`。对于自定义类型，是否使用 `const` 修饰符存在细微差别，但通常情况下我们使用不带 `const` 的形式。例如：

```
1 string name( )      { return "面向对象程序设计"; }    //形式 1
2 const string name( ) { return "面向对象程序设计"; }    //形式 2
```

在上面的形式 2 中返回的是一个常量型的字符串对象，即不得修改该返回对象，但返回的对象是一个新对象，是否修改不会影响到原对象。所以我们更多的是采用形式 1。

第三种，按指针返回。例如 `T* func()`，这等价于 `T* const func()`，但不等价于 `const T* func()`。函数返回时可以返回空指针 `nullptr` 或非空指针。若返回非空指针，务必确保其是一个有效指针，即指向的对象有效。例如：

```
1 T * func( ) {
2     T aT;
3     static T sT;
4     return &aT;    // 非法，虽然部分编译器可以编译通过
5     return nullptr; // 合法
6     return &sT;    // 合法
7 }
```

第四种，按引用返回。例如 `T& func()`，这不等价于 `const T& func()`。由于引用必须对应到一个实际存在的对象，因此返回的对象或变量必须是有效存在的。例如：

```

1  int global = 100; //全局变量
2  int & other(int &, const int & );
3  int & f(int a, int & b, const int & c) {
4      int localVar = 88;          //局部变量
5      static int sVar = 99;       //局部静态量
6      return a;                   // 非法
7      return b;                   // 合法
8      return c;                   // 非法
9      return global;              // 合法
10     return global + 8;          // 非法
11     return localVar;            // 非法
12     return sVar;                // 合法
13     return sVar+2;              // 非法
14     return sVar += 3;           // 合法
15     return other(localVar, b);  // 合法
16     return other(sVar + 2, global); // 非法
17 }
```

由于按引用返回的函数，其返回值对应一个有效对象或变量，因此该函数既能作为右值表达式使用，也能作为左值表达式使用。需要注意的是，返回常量引用和非常量引用在语义上存在差异。例如：

```

1  int      max21(int a,int b)      { return a>b?a:b; }
2  int &     max22(int & a,int& b)   { return a>b?a:b; }
3  const int & max23( int & a,int & b) { return a>b?a:b; }
4  const int & max24(const int & a, const int & b) { return a>b?a:b; }
5  int main( ) {
6      int a = 3;
7      int b = 4;
8      int m1 = max21(a,b);
9      cout << a << b << m1 << endl; //输出 344
10     int m2 = max22(a,b);
11     cout << a << b << m2 << endl; //输出 344
12     max22(a,b) = 5;
13     m2 = max22(a,b);
14     cout << a << b << m2 <<endl; //输出 355
15     int m3 = max23(a,10);          //非法
16     max23(a,b) = 6;                //非法
17     int m3 = max23(a,b);
18     cout << a << b << m3 << endl; //输出 355
```

```

19     max24(a,b) = 7;           //非法
20     int m4 = max24(a,b);
21     cout << a << b << m4 <<endl; //输出 355
22 }

```

而且，不同于按值返回，返回常量引用或变量引用具有不同的语义含义和应用价值。在设计和使用时，应根据具体的模型和函数实现方式，选择合适的返回形式。例如：

```

1  X1  getName1( ) {
2      string name("面向对象程序设计");
3      return name;
4  }
5  X2  getName2( ) {
6      const string name("面向对象程序设计");
8      return name;
9  }
10 X3  getName3( ) {
11     static string name("面向对象程序设计");
12     return name;
13 }
14 X4  getName4( ) {
15     static const string name("面向对象程序设计");
16     return name;
17 }

```

这四个函数的具体实现各不相同，X1、X2、X3 和 X4 作为返回类型时，各有不同的最佳形式。对于 X1 和 X2，最合适的返回类型是 `string` 或 `const string`，而 `string&` 是非法的，`const string&` 则通常会引发警告。对于 X3，最理想的选择是 `const string&`，除非需要外部修改字符串内容，否则不应使用 `string&`。虽然 `string` 和 `const string` 在语法上是可行的，但它们的效率较差。对于 X4，最合适的返回类型同样是 `const string&`，不能使用 `string&`，而 `string` 和 `const string` 虽然语法上可以接受，但效率不佳。

在设计函数的返回类型时，还应注意：

- 遵循人们的常规认知和习惯。例如，在函数 `Date tomorrow(const Date& aDate)` 中，返回类型应为按值返回的 `Date` 类型，而不是引用类型的 `Date&` 或 `const Date&`。
- 引用、指针和 `const` 不同组合后的语义差别。例如：

```

1  struct Data { };
2  Data vd { };
3  const Data cd { };
4  Data * get01( ) { return &vd; }           //合法

```

```
5 Data * get02( ) { return &cd; } //非法
6 Data * get03( ) { return nullptr; } //合法
7 Data * const get04( ) { return &vd; } //合法
8 Data * const get05( ) { return &cd; } //非法
9 Data * const get06( ) { return nullptr; } //合法
10 const Data * get07( ) { return &vd; } //合法
11 const Data * get08( ) { return &cd; } //合法
12 const Data * get09( ) { return nullptr; } //合法
13 Data *& get10( ) { return &vd; } //非法
14 Data *& get11( ) { return &cd; } //非法
15 Data *& get12( ) { return nullptr; } //非法
16 Data * const & get13( ) { return &vd; } //合法
17 Data * const & get14( ) { return &cd; } //非法
18 Data * const & get15( ) { return nullptr; } //合法
19 const Data *& get16( ) { return &vd; } //非法
20 const Data *& get17( ) { return &cd; } //非法
21 const Data *& get18( ) { return nullptr; } //非法
22 const Data * const & get19( ) { return &vd; } //合法
23 const Data * const & get20( ) { return &cd; } //合法
24 const Data * const & get21( ) { return nullptr; } //合法
```